COMPLAINT COMPASS · JUDGE Q&A PREP

“How are you going to scale it?”

Lead with one sentence, then go deeper only as far as the judge pushes. Picture first, numbers second, caveats before they ask.

1. The 30-second answer: say this first

> “We scale by adding copies, not by redesigning. Only about 4% of complaints ever reach an LLM, so cost grows in a straight line: around $433 a month in model calls at 10 million complaints. Most of the pipeline has no shared state, so we just run more workers. The one part that does share state, the list of known issues, is already split by business and by area like payments or delivery. So each of those gets its own single worker, and they all run in parallel.”

Then stop and let them pick the thread.

2. If they push: three layers, in this order

Layer 1. Why it is cheap to scale (the economics)

- 96.1% of complaints never reach the LLM. MEASURED
- $4.33 per 100,000 complaints, against $12 to $594 if an LLM reads every one. MEASURED (baseline priced, not run)
- So an LLM outage or rate limit hits only about 4% of traffic.

Layer 2. How it is built to scale (the architecture)

Point to the architecture slide and walk it top to bottom:

- **Stage A** (identify the customer, Jev gate, embed): each complaint is independent, so add workers. One worker handles about 15 a second; ten handle about 13 million a day. PROJECTED
- **Stage B** (match, verify, LLM, new issues): order matters, because new issues are discovered as complaints arrive. So exactly one writer per business × area. Grocery has 7 partitions; 100 businesses would have about 700, all in parallel.
- Postgres with a vector index replaces SQLite.

Layer 3. When we build it (the roadmap)

- Four phases, each unlocked by a gate: shadow pilot passes → load test plus first paid contract → second domain pack in production.
- “We don’t build capacity ahead of evidence.”

3. Likely follow-ups and your one-liners

They ask	You say
“What breaks first?”	“The single writer and SQLite. Both are already in the plan: one writer per partition, and Postgres.”
“What if Jev goes down?”	“Jev is on the critical path today. The fix is a degraded mode: we sort by similarity alone, label it, and re-check when Jev is back.”

“What about a spike, like an outage day?”	“A queue absorbs it. Most spike complaints are the same issue, so they match for free; only about 4% wait on the LLM.”
“One hot partition, like payments?”	“Most of its complaints match at the free step, so one worker keeps up. If it doesn’t, we split it by region.”
“New customer or industry?”	“A new settings file, not new code. Jev takes its labels in each request, so there’s no retraining.”
“Does it get cheaper over time?”	“Yes. Hard cases the LLM or Jev settles become examples, so next time that wording matches for free.”
“Have you load-tested it?”	“No. The throughput figures are worked out from measured response times. A load test is gate 2, before any contract.”

4. Three rules while answering

- **Say “measured” or “projected” every time.** Measured: 0.55 s, 96.1%, $4.33 per 100k. Projected: 15 a second per worker, 13 million a day, $433 at 10 million. Separating them yourself earns trust.
- **Admit the real risk before they find it.** “Scale isn’t our biggest risk; accuracy on real, messy data is, and that’s why our first step is a shadow pilot.” It shows judgement and moves the talk onto ground you’ve prepared.
- **Point at a slide or diagram, don’t recite.** Architecture slide for “how”, the cost-at-scale slide for “how much”, the roadmap for “when”.

5. Close the answer with

> “So: the economics are measured, the architecture is a standard pattern, and we only build each phase once the last one has proven itself.”

Sources for the numbers: live run report (backend/runs/grocery_report.json), HANDOFF.md §4, SCALABILITY.md, and the Market and Scale report.